

OPPORTUNITIES CMS

Developer Requirements & Functional Specification

Purpose

Build a dynamic opportunities system where the client can create and manage any opportunity from an admin panel, and the public website updates automatically.

- Dynamic opportunity creation, editing, viewing, deleting and publishing
- Admin-uploaded opportunity image used automatically on the frontend
- Category, location and attribute filters
- 12 opportunities per page with automatic pagination
- Each opportunity has its own Apply Now URL
- Frontend detail page driven by the same reusable data model

Reference screenshots provided by client/user are included in this document to clarify the intended frontend opportunity experience.

1. System Overview

The Opportunities section should be a database-driven CMS. The admin panel is the source of truth; the public Opportunities page reads the published records and renders them using reusable templates.

Core flow

Admin	Backend database	Frontend	Applicant
Create / edit / publish	Store opportunity + image + attributes	Search, filters, pagination, details	Apply Now and open external application

Important: the admin should not manually create frontend pages for each opportunity. One reusable opportunity template is enough.

2. Public Opportunities Page

The current Opportunities page should remain the main discovery page. Its cards and detail pages become dynamic and are populated from the admin-managed database.

Recommended frontend behavior

- Show published opportunities only. Draft/unpublished records remain invisible to public users.
- Display opportunity cards using the admin-provided title, image, category, location and selected summary fields.
- Clicking a card opens a reusable detail page for that specific opportunity.
- Provide search plus dynamic filters that operate on database fields.
- Keep the UI consistent regardless of whether the opportunity is a job, scholarship, internship, grant or another category.

Figure 1 - Current/reference Opportunities page supplied by the client.

3. Admin Panel - Opportunities Management

Create an Opportunities module in the admin panel. The client should be able to manage the complete lifecycle of every opportunity without developer involvement.

- List all opportunities with search, filters and status.
- Create a new opportunity.
- Open/read a full opportunity record.
- Edit any existing opportunity.
- Delete an opportunity (prefer confirmation and soft-delete if appropriate).
- Publish or unpublish an opportunity.
- Upload/replace the opportunity image.
- Control the application link used by the Apply Now button.

Admin list example

Opportunity	Category	Location	Status	Posted	Actions
Retail Assistant	Employment	New York	Published	2 days ago	View / Edit / Delete
Scholarship 2026	Scholarship	Online	Draft	-	View / Edit / Publish
Marketing Intern	Internship	Chicago	Published	5 days ago	View / Edit / Delete

4. Opportunity Creation / Edit Form

Every field below should be available to the admin as appropriate. Use structured fields for filters and consistent UI; use rich text fields/sections for long-form content.

Field	Type / Input	Frontend use
Title	Text	Main heading / card title
Category	Select / taxonomy	Filter + category label
Opportunity type	Select	Full-time, part-time, internship, scholarship, etc.
Location	Structured text: country / region / city as needed	Location filter + detail page
Organization / provider	Text	Detail page
Opportunity image	Image upload	Card + detail page hero/image
Short description	Textarea / rich text	Card or preview
Salary / package	Text or structured min/max + currency	Detail page + optional filter
Experience required	Select or text	Detail page + optional filter
Education / qualification	Text / rich text	Detail page

Field	Type / Input	Frontend use
Posted date	Auto/manual date	Sort + “posted X days ago”
Application deadline	Date (optional)	Detail page
Apply Now URL	URL input	Apply Now button destination
About the role / opportunity	Rich text	Detail page section
Requirements	Rich text / list	Detail page section
How to apply	Rich text	Detail page section
Status	Draft / Published / Unpublished	Visibility control

Image requirement: the admin uploads the image while creating/editing the opportunity. The uploaded asset is stored against that opportunity record and is automatically rendered on the frontend.

5. Opportunity Detail Page

Use one reusable detail-page template. The content changes based on the selected opportunity record.

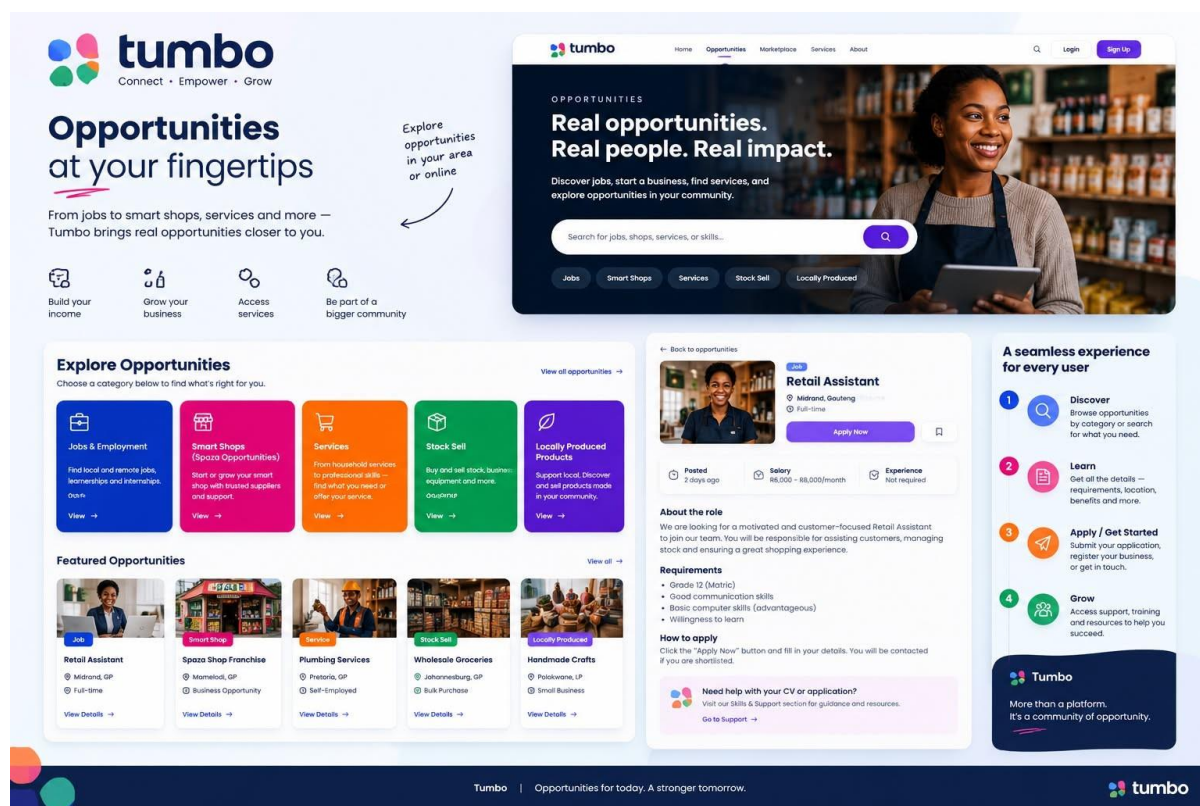


Figure 2 - Client-provided reference for the desired opportunity detail experience.

Expected structure

- Opportunity image
- Title + category

- Location and employment/opportunity type
- Posted date / time since posted
- Salary/package where provided
- Experience required where provided
- Apply Now button
- About the role / opportunity
- Requirements
- How to apply
- Any additional fields that were populated by the admin

Apply Now behavior: the button must read the URL stored on that opportunity. Clicking it should redirect the user to that exact external application destination. Do not hard-code one application URL for every opportunity.

6. Filtering, Search & Sorting

Filters should be database-driven. The most important filter is Location. The user should be able to select a location and see only matching published opportunities.

Location filter behavior

- Allow users to search/select a city, such as New York.
- Where useful, support country -> region/state -> city so locations remain organized.
- Only show locations that exist in the published opportunity dataset, or provide a searchable location control.
- If no opportunities match, show a clear empty state such as “No opportunities found for this location.”
- Filters should work together with search and other filters rather than replacing the whole result set.

Recommended filter set

Filter	Examples
Location	New York, Chicago, London, Online
Category	Employment, Scholarship, Internship, Grant, Training, Other
Opportunity type	Full-time, Part-time, Contract, Internship, Program
Experience	No experience, 1-2 years, 3-5 years, etc.
Salary/package	Optional range or text filter, depending on implementation
Date posted	Newest, last 24 hours, last 7 days, etc. (optional)

Recommended default sorting: newest published opportunities first, so new opportunities naturally appear on Page 1.

7. Pagination - Exactly 12 Results Per Page

Pagination is automatic and database-driven. The public page displays a maximum of 12 published opportunities per page.

Required rule

- Opportunities 1-12 -> Page 1
- When opportunity 13 exists -> Page 2 is automatically available and contains opportunity 13 (plus any later records until it reaches 12).
- Opportunities 13-24 -> Page 2
- Opportunities 25-36 -> Page 3
- Opportunities 37-48 -> Page 4
- Continue the same pattern indefinitely. The final page may contain fewer than 12 items until it becomes full.
- Admin does not manually create or manage pages.

Example: 31 published opportunities = Page 1 (12) + Page 2 (12) + Page 3 (7). Page 3 still exists; it does not wait for 36 records to be created.

Recommended controls

Changing a filter should reset pagination to Page 1 and calculate pages from the filtered result set.

8. Suggested Data Model

The exact schema can follow the project stack, but the following fields cover the requested functionality.

```
Opportunity { id title slug categoryId / category opportunityType organization country regionOrState city
locationLabel imageUrl shortDescription salaryText or salaryMin/salaryMax + currency experienceRequired
educationRequired postedAt applicationDeadline (optional) applyUrl aboutContent requirementsContent
howToApplyContent status: DRAFT | PUBLISHED | UNPUBLISHED createdAt updatedAt }
```

Use a separate Category table/taxonomy when categories need to be administered dynamically. Store the image in the project storage/media service and save its URL/reference on the opportunity record.

9. Suggested Backend/API Behavior

Operation	Purpose
GET /opportunities?page=1&limit=12	Published opportunities for a page
GET /opportunities?location=New%20York&page=1&limit=12	Filtered opportunities
GET /opportunities/:id or /:slug	Single opportunity detail
POST /admin/opportunities	Create
PUT/PATCH /admin/opportunities/:id	Edit
DELETE /admin/opportunities/:id	Delete
PATCH /admin/opportunities/:id/status	Publish/unpublish
POST /admin/media or equivalent	Upload opportunity image

Server-side pagination is preferred so the system remains efficient when the number of opportunities becomes large. The API should return total count / total pages so the frontend can render pagination controls accurately.

10. Validation, Permissions & Reliability

- Only authenticated/authorized admins can create, edit, delete or publish opportunities.
- Apply Now URL must be a valid URL; reject malformed or unsafe input.
- Image upload should validate file type/size and generate an optimized web image where appropriate.
- Title and category should be required; define required/optional fields clearly.
- Draft records should never appear publicly.
- Escape/sanitize rich text according to the stack to prevent XSS.
- Confirm deletion, and preferably soft-delete records if audit/recovery is needed.
- When a record is edited, the frontend should reflect the latest saved data without requiring code changes.
- Preserve accessibility: meaningful alt text for opportunity images and keyboard-accessible filters/buttons.

11. Acceptance Checklist

- Admin can log in and access Opportunities management.

- Admin can create a job, scholarship, internship or other opportunity using the same form.
- Admin can upload an image during creation and the image appears on the frontend.
- Admin can edit every important field and changes appear on the public page.
- Admin can publish/unpublished and only published records are public.
- Admin can delete an opportunity.
- Each opportunity has its own Apply Now URL and button redirects to that URL.
- Location filter can return opportunities for a selected city such as New York when matching records exist.
- Filters can be combined with search.
- Public results are paginated at 12 opportunities per page.
- Page 2 automatically becomes available once the 13th result exists.
- The final page can contain fewer than 12 items.
- Detail pages are generated from one reusable opportunity template.
- No frontend opportunity page needs to be manually coded per listing.

End of specification - this document describes the requested behavior, not a requirement to use a particular framework or database.